

Build Tools Vite & Webpack

Bundle, Optimise, and Ship Your App

■ LEARNING OUTCOME

By the end of this module you will configure Vite for development and production, understand what bundlers do, use environment variables safely, optimise bundle size, and set up CI/CD deployment.

■ Skills You Will Gain

- Understand what build tools do and why they exist
- Configure Vite for development and production
- Use environment variables safely
- Analyse and optimise bundle size
- Configure proxy for API development
- Set up CI/CD with GitHub Actions and Vercel

■ Build tools transform your JSX/TypeScript into browser-compatible files. Without them, browsers cannot run React code. Knowing your build tool separates developers who can debug production from those who cannot.

What The Bundler Does

Your source: import React from 'react', .jsx files, .module.css, images\n\nThe bundler:\n1. Module resolution: Follow imports, build dependency graph\n2. Transformation: JSX->JS, TypeScript->JS, CSS Modules->JS\n3. Tree shaking: Remove dead code (unused imports)\n4. Code splitting: Split into chunks for lazy loading\n5. Minification: Remove whitespace, shorten names\n6. Asset handling: Hash filenames for long-term caching\n\nOutput: dist/ with index.html, main.abc123.js, chunk.xyz.js\nVite: native ESM in dev (instant start), Rollup for production.\nWebpack: bundles everything even in dev -- slower but more plugins.

SECTION 1

Vite Configuration

```
// vite.config.ts import { defineConfig } from 'vite'; import react from '@vitejs/plugin-react';
import path from 'path'; export default defineConfig({ plugins: [react()], resolve: { alias: { '@':
path.resolve(__dirname, './src') }, // Now: import Button from '@components/Button' // Not: import
Button from '.././components/Button' }, server: { port: 5173, proxy: { '/api': { target:
'http://localhost:3000', changeOrigin: true, rewrite: p => p.replace(/^\/api/, ''), }, }, }, build:
{ outDir: 'dist', rollupOptions: { output: { manualChunks: { 'react-vendor':
['react','react-dom','react-router-dom'], 'query-vendor': ['@tanstack/react-query'], }, }, },
chunkSizeWarningLimit: 800, }, }); # ENVIRONMENT VARIABLES # .env -- always loaded # .env.local --
gitignored, local overrides # .env.production -- production only # Only VITE_ prefix is exposed to
client code! # .env # VITE_API_URL=https://api.example.com # VITE_APP_NAME=My App #
SECRET_KEY=never-exposed (no VITE_ prefix) // Access in React const apiUrl =
import.meta.env.VITE_API_URL; const isDev = import.meta.env.DEV; const isProd =
import.meta.env.PROD; # GitHub Actions CI/CD (.github/workflows/ci.yml) # on: push to main # steps:
checkout, setup-node, npm ci, npm run lint, npm test, npm run build # deploy: vercel-action with
VERCEL_TOKEN secret
```

	How	Impact
	lazy() + Suspense for routes	Load only needed code per route
	Named imports (auto in Vite/ESM)	Remove unused library code
n	WebP format, set width/height, lazy	Huge size reduction
	manualChunks in rollupOptions	Better browser caching headers
	rollup-plugin-visualizer	Find what bloats your bundle
	Vercel/Netlify serve from edge CDN	Global low latency

■ PRACTICE Q & A

Q1. What is the difference between Vite and Webpack?

A: Vite uses native ES modules in dev (no bundling -- instant start), Rollup for production. Webpack bundles everything even in dev -- slower start but more mature plugins. Vite recommended for new projects.

Q2. What is tree shaking?

A: Dead code elimination. Bundlers remove exports not imported anywhere. Works with ES modules (import/export). import {useState} from 'react' only includes useState, not all of React.

Q3. What is code splitting?

A: Splitting bundle into chunks loaded on demand. `lazy()` => `import('./AdminPanel')` only downloads AdminPanel when user visits /admin. Reduces initial bundle size and Time to Interactive.

Q4. Why prefix env variables with `VITE_`?

A: Security: only `VITE_` prefixed variables are embedded in client bundle. `SECRET_API_KEY` without prefix stays server-side. The prefix is a deliberate decision to expose the variable publicly.

Q5. What is the `dist/` folder?

A: Production build output from `npm run build`. Contains: hashed JS chunks, CSS, optimised images, `index.html`. Hash in filenames enables long-term browser caching.

■ Build Tools Cheat Sheet	
<code>npm run dev</code>	Start Vite dev server
<code>npm run build</code>	Production build -> <code>dist/</code>
<code>npm run preview</code>	Preview production locally
<code>import.meta.env.VITE_X</code>	Access env variable
<code>VITE_</code> prefix required	To expose var to client
<code>resolve.alias: {'@':'src'}</code>	Path alias
<code>server.proxy: {'/api':target}</code>	Proxy dev API calls
<code>manualChunks: {vendor:['react']}</code>	Manual code splitting
<code>lazy(()=>import('./Page'))</code>	Dynamic import
<code>.env.local</code> (gitignored)	Local secrets
<code>dist/</code>	Production build output
<code>rollup-plugin-visualizer</code>	Bundle size analysis